IFPB - Campus Campina Grande
Programação Orientada a Objetos - Checkpoint 04

Nome: Nota:

QUADRO DE RESPOSTAS

Q1:	Q2:	Q3:	Q4:	Q5:	Q6:

Q7				
1)	2)	3)	4)	
5)	6)	7)	8)	

CENÁRIO BASE

Estadia.java

```java
public class Estadia {
  private int horas;
  private boolean encerrada;
  private Cobranca cobranca;
  public Estadia() {
    horas = 0;
    encerrada = false;
  }
  public int getHoras() { return horas; }
  public boolean isEncerrada() {
    return encerrada;
  }
  public void adicionarHoras(int qtd) {
    if (!encerrada && qtd > 0) {
      horas += qtd;
    }
  }
  public void encerrar(Cobranca opcao) {
    if (!encerrada && opcao != null) {
      cobranca = opcao;
      encerrada = true;
    }
  }
  public double calcularValor() {
    if (!encerrada) {
      return 0.0;
    }
    return cobranca.calcularValor(this);
  }
}
```

CobrancaPorHora.java

```java
public class CobrancaPorHora
    implements Cobranca {
  public double calcularValor(Estadia e){
    return e.getHoras() * 5.0;
  }
}
```

CobrancaPorDiaria.java

```java
public class CobrancaPorDiaria
    implements Cobranca {
  public double calcularValor(Estadia e){
    return 40.0;
  }
}
```

Cobranca.java

```java
public interface Cobranca {
  double calcularValor(Estadia estadia);
}
```

Ajustavel.java

```java
public interface Ajustavel {
  boolean alterarTarifa(double valor);
}
```

Flexivel.java

```java
public class Flexivel
    implements Cobranca, Ajustavel {
  private double tarifa = 5.0;
  private int saldo;
  public Flexivel(int s) { saldo = s; }
  public boolean alterarTarifa(
      double v) {
    if (v<=0 || saldo==0) return false;
    tarifa = v;
    saldo--;
    return true;
  }
  public int getSaldo() { return saldo; }
  public double calcularValor(Estadia e){
    return e.getHoras() * tarifa;
  }
}
```

Nesta avaliação, cada questão parte do cenário base; alterações valem apenas para a própria questão.

QUESTÕES

Q1. (10 pontos) O trecho abaixo percorre uma lista com duas modalidades de cobrança. Considere os imports de `List` e `ArrayList` corretos.

```
Estadia e = new Estadia();
e.adicionarHoras(4);
List<Cobranca> opcoes = new ArrayList<>();
opcoes.add(new CobrancaPorHora());
opcoes.add(new CobrancaPorDiaria());
for (Cobranca c : opcoes) {
  c.calcularValor(e);
}
```

Por que a mesma chamada, `c.calcularValor(e)`, executa cálculos diferentes nas duas passagens pelo laço?

A) O compilador usa a classe de cada elemento para alterar o tipo declarado de `c`.

B) O tipo de `c` permite a chamada, e o objeto referenciado determina a implementação.

C) O objeto `e` escolhe a cobrança, e `c` encaminha a chamada para essa escolha.

D) O tipo de `c` fixa a implementação, e os atributos de cada cobrança mudam o resultado.

E) O compilador define a implementação de cada passagem a partir das chamadas a `add`.

Q2. (10 pontos) A classe `Fixa` registra quantas vezes seu método de cálculo foi chamado. `Taxa` usa uma cobrança base e acrescenta R$ 4 ao valor.

```
public class Fixa implements Cobranca {
  private int vezes;
  public double calcularValor(Estadia e) {
    vezes++;
    return 12.0;
  }
  public int getVezes() { return vezes; }
}
```

```
public class Taxa implements Cobranca {
  private Cobranca base;
  public Taxa(Cobranca c) { base = c; }
  public double calcularValor(Estadia e) {
    double valor = base.calcularValor(e);
    if (valor == 0.0) return 4.0;
    return base.calcularValor(e) + 4.0;
  }
}
```

Em seguida, executamos:

```
Fixa base = new Fixa();
Cobranca c = new Taxa(base);
double valor = c.calcularValor(
    new Estadia());
```

Ao final desse trecho, quais são os valores de `valor` e `base.getVezes()`, respectivamente?

A) `12.0` e `1`.

B) `16.0` e `1`.

C) `16.0` e `2`.

D) `20.0` e `2`.

E) `28.0` e `2`.

Q3. (10 pontos) A classe `Recibo` permite calcular a cobrança e emitir um comprovante. A cobrança por hora continua como no cenário base.

```
public interface Comprovavel {
  String emitir();
}
```

```
public class Recibo
    implements Cobranca, Comprovavel {
  public double calcularValor(Estadia e) {
    return 25.0;
  }
  public String emitir() { return "OK"; }
}
```

Considere o trecho:

```
Cobranca primeira = new Recibo();
Cobranca segunda = new CobrancaPorHora();
if (primeira instanceof Comprovavel) {
  Comprovavel a = (Comprovavel) primeira;
  System.out.println(a.emitir());
  Comprovavel b = (Comprovavel) segunda;
  System.out.println(b.emitir());
}
```

O trecho compila? Se compilar, o que acontece ao executá-lo?

A) Compila, mas não imprime: a condição do `if` é falsa.

B) Compila e imprime `OK` duas vezes, sem falhar.

C) Não compila: `Cobranca` não declara o método `emitir`.

D) Compila, mas falha no cast de `primeira`, antes de imprimir.

E) Compila, imprime `OK` uma vez e falha no cast de `segunda`.

Q4. (10 pontos) Um mesmo objeto pode ser referenciado pelos tipos `Cobranca` e `Cancelavel`, como no exemplo abaixo.

```
public interface Cancelavel {
  void cancelar();
}
```

```
public class Evento
    implements Cobranca, Cancelavel {
  private boolean cancelada;
  public void cancelar() {
    cancelada = true;
  }
  public double calcularValor(Estadia e) {
    if (cancelada) return 0.0;
    return 30.0;
  }
}
```

```
Evento evento = new Evento();
Cobranca c = evento;
Cancelavel x = evento;
Estadia e = new Estadia();
```

Cada chamada a seguir deve ser analisada separadamente, após as declarações acima. Quais compilam sem acrescentar casts?

```
x.cancelar();          // I
x.calcularValor(e);    // II
e.encerrar(x);        // III
e.encerrar(c);        // IV
```

A) Apenas I, II e IV.

B) Apenas I e IV.

C) Apenas I.

D) Apenas I, III e IV.

E) I, II, III e IV.

Q5. (10 pontos) A classe `Registro` guarda a estadia recebida no cálculo. Analise também a implementação de `Servico`.

```
public class Registro
    implements Cobranca {
  private Estadia alvo;
  public double calcularValor(Estadia e) {
    alvo = e;
    return 12.0;
  }
  public Estadia getAlvo() { return alvo; }
}
```

```
public class Servico implements Cobranca {
  private Cobranca base;
  public Servico(Cobranca c) { base = c; }
  public double calcularValor(Estadia e) {
    Estadia outra = new Estadia();
    return base.calcularValor(outra) + 3.0;
  }
}
```

O teste verifica se a cobrança base recebeu a mesma estadia que solicitou o cálculo. Considere os imports de JUnit corretos.

```
@Test
void teste() {
  Estadia e = new Estadia();
  Registro registro = new Registro();
  e.encerrar(new Servico(registro));
  e.calcularValor();
  assertTrue(registro.getAlvo() == e);
}
```

O teste passa ou falha? Assinale o resultado e a justificativa corretos.

A) Passa: o cálculo usa a cobrança `registro`, preservando também a estadia original.

B) Passa: o teste compara o conteúdo dos objetos, e as duas estadias têm zero horas.

C) Falha: `Registro` armazena `outra`, que é uma instância diferente da estadia `e`.

D) Falha: o cálculo não chama `Registro`, pois a estadia não tem horas registradas.

E) Falha: `Registro` recebe `e`, mas a atribuição `alvo = e` cria uma cópia da estadia.

Q6. (10 pontos) A classe `Seguro` acrescenta R$ 7 ao valor da cobrança recebida no construtor.

```
public class Seguro implements Cobranca {

  private Cobranca base;
  public Seguro(Cobranca c) { base = c; }
  public double calcularValor(Estadia e) {
    return base.calcularValor(e) + 7.0;
  }
}
```

Uma cobrança com seguro é criada e, depois, a variável `base` recebe outra modalidade:

```
Estadia e = new Estadia();
e.adicionarHoras(3);
Cobranca base = new CobrancaPorHora();
Cobranca segurada = new Seguro(base);
base = new CobrancaPorDiaria();
e.encerrar(segurada);
```

Qual valor é devolvido por `e.calcularValor()` ao final desse trecho?

A) `0.0`.

B) `15.0`.

C) `40.0`.

D) `22.0`.

E) `47.0`.

Q7. (40 pontos) Use `Ajustavel` e `Flexivel` do cenário base e as duas classes abaixo.

```
public class Adicional implements Cobranca
{
  private Cobranca base;
  private double adicional;
  public Adicional(Cobranca c, double v) {
    base = c;
    adicional = v;
  }
  public double calcularValor(Estadia e) {
    return base.calcularValor(e)
        + adicional;
  }
}
```

```
public class Teto implements Cobranca {
  private Cobranca base;
  private double limite;
  public Teto(Cobranca c, double max) {
    base = c;
    limite = max;
  }
  public double calcularValor(Estadia e) {
    double valor = base.calcularValor(e);
    if (valor > limite) return limite;
    return valor;
  }
}
```

Os cinco trechos são **independentes**. Escreva as oito saídas na ordem dos `println` numerados. Cada saída vale **5 pontos**. Considere os imports de `List` e `ArrayList` corretos.

Trecho I

```
Estadia e = new Estadia();
e.adicionarHoras(2);
Cobranca base = new CobrancaPorHora();
Cobranca uma = new Adicional(base, 3.0);
Cobranca duas = new Adicional(uma, 4.0);
e.encerrar(duas);
System.out.println(e.calcularValor()); //1
```

Trecho II

```
Flexivel f = new Flexivel(3);
List<Ajustavel> opcoes = new ArrayList<>();
opcoes.add(f);
opcoes.add(f);
for (Ajustavel a : opcoes) {
  a.alterarTarifa(6.0);
}
System.out.println(f.getSaldo());      //2
```

Trecho III

```
Estadia e = new Estadia();
Cobranca base = new CobrancaPorDiaria();
Cobranca a = new Teto(
    new Adicional(base, 6.0), 35.0);
Cobranca b = new Adicional(
    new Teto(base, 35.0), 6.0);
double diferenca =
  b.calcularValor(e) - a.calcularValor(e);
System.out.println(diferenca);         //3
```

Trecho IV

```
Estadia e = new Estadia();
e.adicionarHoras(7);
Cobranca opcao = new CobrancaPorHora();
if (!(opcao instanceof Ajustavel)) {
  opcao = new CobrancaPorDiaria();
}
System.out.println(
    opcao.calcularValor(e));           //4
opcao = new Teto(
    new CobrancaPorHora(), 20.0);
System.out.println(
    opcao.calcularValor(e));           //5
```

Trecho V

```
Flexivel f = new Flexivel(2);
f.alterarTarifa(-2.0);
System.out.println(f.getSaldo());      //6
Flexivel g = new Flexivel(1);
g.alterarTarifa(4.0);
System.out.println(
    g.alterarTarifa(6.0));             //7
Cobranca c = new Adicional(g, 3.0);
System.out.println(
    c instanceof Ajustavel);           //8
```